Práctica 1. Organización de ficheros

Autor: Lucas Hidalgo Herrera

Grado: Doble Grado en Ingeniería Informática y Matemáticas

Asignatura: Administración de Bases de Datos

Profesor: Ignacio José Blanco Medina

Fecha: 23 de marzo de 2026

Índice General

1 Introducción **1**

2 Estructuras soporte **1**

3 TDA fileRecord **2**
3.1 Estructura en memoria 2
3.2 Operaciones públicas 3
3.3 Operaciones Privadas 4

4 TDA fileBlock **5**
4.1 Estructura en memoria 5
4.2 Operaciones públicas 5
4.3 Operaciones privadas 8

5 TDA fileHeader **10**

6 TDA fileASF **10**
6.1 Estructura en memoria 10
6.2 Operaciones privadas 11
6.3 Operaciones públicas 16

1 Introducción

A lo largo de este documento se va a realizar el trabajo 1 de la asignatura de *Administración de Bases de Datos*. Dicho trabajo consiste en documentar, diseñar y especificar algunos tipos de datos abstractos para el trabajo con ficheros de carácter secuencial físico.

Estos tipos de datos abstractos están basados en la estructura en memoria de los ficheros de antiguos sistemas operativos; que como vimos en clase, no es más que una secuencia de bloques junto a una cabecera, donde cada bloque es una secuencia de registros precedidos globalmente por una cabecera.

De cada uno de ellos, trataremos la *estructura en memoria*, las *operaciones públicas* y las *operaciones privadas* [1]. Por tanto, debemos especificar y diseñar los tipos de datos abstractos **TDA fileASF**, **TDA fileHeader**, **TDA fileBlock** y **TDA fileRecord**.

2 Estructuras soporte

Con vistas a trabajar con mayor facilidad vamos a definir una serie de estructuras que trataremos como una sola. Esto es causa de que son estructuras unificadas para todos los objetos de un mismo tipo de dato abstracto.

Estas estructuras serán dos:

1. *unifiedRecord*: esta estrucutra refleja el conjunto de atributos del tipo de dato abstracto *TDA fileRecord* que están unificados para todos los objetos del mismo .

   Esta estructura está compuesta por los atributos *headerRecordSize*, *numAttr*, *tipoCampos* y *camposSize*; es decir, es la siguiente

```
// Representación en memoria del registro
private:
estructura unifiedRecord {
    entero headerRecordSize;
    entero numAttr;
    secuencia<entero> tipoCampos;
    secuencia<entero> camposSize;
}
```

Listing 1: Estructura *unifiedRecord*

2. *unifiedBlock*: esta estrucutra refleja el conjunto de atributos del tipo de dato abstracto *TDA fileBlock* que están unificados para todos los objetos del mismo .

   Esta estructura está compuesta por los atributos *headerBlockSize* y *blockSize*; es decir, es la siguiente

[1] Todo ello en pseudocódigo

```
// Representación en memoria del registro
private:
estructura unifiedBlock {
    entero headerBlockSize;
    entero blockSize;
}
```

Listing 2: Estructura *unifiedRecord*

Cabe destacar que estas estructuras no son estrictamente necesarias, simplemente se han creado para facilidad de comprensión. Entenderemos que los operadores de igualdad hacen copia parámetro a parámetro; así como que su creación se hace atributo por atributo, siempre que la visibilidad del objeto en el que se encuentra lo permita.

Además, debo comentar que los atributos no son constantes puesto que como estructura no es necesario; sin embargo, una vez formen parte de uno de los tipos de datos abstractos que conforman el fichero sí pasarán a ser constantes.

Estas estructuras ayudarán a comprender que, en la situación en la que estamos (bloques homogéneos con registros de longitud fija), esta estructura se impone al principio de forma conjunta y no se puede manipular.

3 TDA fileRecord

Comenzamos por el más interno, lo que ayudará a aprender de forma constructiva el objetivo de la práctica.

3.1 Estructura en memoria

Siguiendo el guión de la práctica, un registro es una secuencia de bytes que almacena dos aspectos, el primero de ellos es la estructura propia del registro y el segundo son los datos siguiendo la estructura que se ha impuesto en el primer dato.

```
// Representación en memoria del registro
private:
estructura fileRecord {
    const unifiedRecord structRecord;
    secuencia<bytes> content;
}
```

Listing 3: Especificación de la estructura del registro en memoria

Como vemos, el atributo *structRecord* que almacena toda la estructura del registro y que es idéntica en todos los de su mismo tipo, es constante. Esto pone de manifiesto que se trabajará como un todo y que no podrá modificarse una vez creado el objeto.

En adición, es conveniente recordar que esta estructura vendrá reflejada en la cabecera del archivo y deberán heredarla del mismo al ser creados para mantener la estructura homogénea de los bloques con registros de longitud fija.

3.2 Operaciones públicas

Puesto que uno de los requisitos que se nos impone en la práctica es que el contenido de los registros pueda ser obtenido por un objeto de tipo *TDA fileASF*, debemos imponer algunas de las funciones de tipo *get* y tipo *set*.

```
// Getter y Setters
public:
/**
* Getter para obtener la estructura del registro
*
* @ret: copia de la estructura del registro
**/
unifiedRecord getStructRecord() const;

/**
* Getter para obtener el contenido de todo el registro
*
* @ret: copia del contenido del registro
**/
secuencia<bytes> getContent() const;

/**
* Setter para cambiar el contenido de todo el registro
*
* @param newContent: secuencia de bytes que alberga el nuevo
    contenido del registro
*
* @pre: content.size()==newContent.size
* @pos: No se cambia el tamaño del registro, pero sí su contenido
**/
void setContent(secuencia<bytes> newContent);
```

Listing 4: Operaciones públicas del registro

Con respecto al único *setter* que he creado, he supuesto que los datos del nuevo contenido tienen el mismo tipo; aún así, si no lo fuera, no se debería realizar la operación.

Entonces, en caso de que fuera necesario, se podría imponer la condición *newContent[i].type==tipoCampos[i]*, pero no soy muy partidario de este tipo de prácticas.

Realmente, ese *setter* no es correcto al completo, en una estructura del tipo que trabajamos la modificación de los registros está prohibidísima [2]; sin embargo, como tipo de dato abstracto puede resultar útil.

Como aclaración, el motivo por el cual todos los *getter* devuelven una copia es para evitar posibles modificaciones por referencia dependiendo del lenguaje de programación que se use; verificando así que no se pueden modificar datos que no se deban modificar.

Por último, debemos crear un constructor y un destructor de este tipo de dato.

```
// Constructor y destructor del registro
```

[2] Se marca como borrado y se crea otro con la misma clave y los cambios necesarios.

```
public:
/**
* Crea una instancia del tipo fileRecord con los datos pasados como
    argumentos
* @param structRecord: estructura del registro a crear
* @param cont: contenido de cada uno de los campos del registro
*
* @pre: structRecord.headerSize>0, structRecord.numAttr>0
* @pos: Se crea una instancia de tipo fileRecord
**/
fileRecord (unifiedRecord structRecord, secuencia<bytes> content);

/**
* Constructor de copia
* @param otherFileRecord: registro a copiar
*
* @pre: otherFileRecord != null
* @pos: Se crea una instancia de tipo fileRecord idéntica a
    otherFileRecord
**/
fileRecord (fileHeader otherFileRecord);

/**
* Destruye una instancia de fileRecord
*
**/
~fileRecord()
```

Listing 5: Constructor y destructor del registro

3.3 Operaciones Privadas

Como cabia de esperar, considero que este tipo de dato no admite operaciones privadas debido a que no dispone de una funcionalidad activa. Es decir, es claro que la principal funcionalidad de este tipo de dato es portar información de memoria a disco y de disco a memoria.

Por tanto, desde mi punto de vista, las operaciones privadas exigidas para el manejo del disco se delegan al tipo de dato abstacto *TDA fileASF* o *TDA fileBlock*.

No obstante, podríamos pensar en una operación privada que realice una de las comprobaciones más importantes, la verificación de tamaños en la creación de una instancia de este tipo de dato; tendría la siguiente especificación:

```
// Operaciones privadas
private:
/**
* Comprueba las restricciones de que los tamaños y cantidades sean
    positivos así como que el tamaño del contenido sea igual a la
    suma de los tamaños especificados en camposSize al estar en un
    ASF. Además verifica que haya tantos tipos de campos como numero
     de atributos especificados. En definitiva, realiza las
    comprobaciones necesarias para no crear un objeto incorrecto.
*
* @param structRecord: estructura del registro
```

```
* @param cont: contenido de cada uno de los campos del registro
*
* @ret: true -> tamaños positivos y dentro de las restricciones de tipos
**/
boolean checkSizes (unifiedRecord structRecord, secuencia<bytes> content);
```

Listing 6: Operaciones privadas del registro

4 TDA fileBlock

Está claro que un fichero es un conjunto de registros. Sin embargo, la memoria no trabaja con registros como tal, sino que trabaja con bloques, cada uno de ellos compuesto por un conjunto de registros.

Con objetivo de modelar con mayor facilidad, y siguiendo las instrucciones del guión del trabajo, vamos a crear la siguiente estructura que modela cómo se guarda cada registro dentro de un bloque.

```
// Representación en memoria del registro
private:
secuencia<bytes> recordBlock {
    entero idRecord;
    boolean stateRecord;
    secuencia<bytes> contentRecord;
}
```

Listing 7: Especificación de la estructura de un registro en un bloque

4.1 Estructura en memoria

Por tanto, usando la estructura definida al inicio de esta sección, tenemos que la estructura en memoria de un bloque de un archivo queda como sigue:

```
// Representación en memoria del registro
private:
estructura fileBlock {
    entero idBlock;
    secuencia<recordBlock> records;
}
```

Listing 8: Especificación de la estructura del bloque del fichero en memoria

4.2 Operaciones públicas

Como todo tipo de dato abstracto, deberemos poder crearlo y destruirlo; puesto que en ocasiones necesitaremos crear un bloque cuyo contenido sea vacío, he decidido permitir esta posibilidad.

```
// Constructor y destructor de bloque
public:
/**
```

```
* Crea una instancia del tipo fileBlock con los datos pasados como argumentos
* @param idBlock: identificador de Bloque
* @param records: registros del bloque
*
* @pre: idBlock>0, records.idRecord[i]>0, records.numElements<=Bfr
* @pos: Se crea una instancia de tipo fileBlock
**/
fileBlock (entero idBlock, secuencia<recordBlock> records);

/**
* Crea una instancia de tipo fileBlock vacía, solo con el idBlock
* @param idBlock: identificador del bloque
*
* @pre: idBlock>0
* @pos: Se crea una instancia de tipo fileBlock vacía
**/
fileBlock (entero idBlock);

/**
* Destruye una instancia de fileRecord
*
* @pos: Se crea una instancia de tipo fileRecord
**/
~fileBlock()
```

Listing 9: Constructor y destructor de bloque

Solo con estas operaciones, nuestro tipo de dato *fileBlock* es un bolso con cartas pero no permite trabajar con ellas, ni obtenerlas, ni añadirlas, ni, tampoco, eliminarlas.

Para ello, crearemos algunas operaciones que hagan este tipo de operaciones. Cabe destacar que, debido a la estructura que manejamos, no debemos permitir el modificado de los registros [3] pues no será más que un borrado y un añadido.

Cada una de estas operaciones actúa de la siguiente manera:

- *addRecord*: se encarga de añadir un registro al final del bloque; podremos avanzar en el bloque gracias a que los registros son de longitud fija y conoceremos el factor de bloqueo.
- *deleteRecord*: independientemente de la versión, se encarga de buscar el registro comenzando desde el principio hasta el final; si encuentra el registro modifica el estado de registro a borrado.

Las operaciones quedan como sigue:

```
// Operaciones públicas de bloque
public:
/**
* Añade un registro al bloque; en caso de que el bloque esté completo devolverá -1
*
```

[3] En *fileRecord* sí lo habíamos permitido; sin embargo, comenzamos ya a ceñirnos a la estructura que vamos a desarrollar.

```
* @param record: registro a añadir
* @param structRecordFile: estructura de los registros del fichero
*
* @pre record.structRecord = structRecordFile
* @pos: se añade el registro al bloque si es posible por tamaño, el regisro dispondrá de su identificador de registro y de estado a true.
*
* @ret: entero que determina el estado de la operación:
*       - (-1) el bloque está completo y se debería crear otro bloque
*       - (0) no se ha podido realizar la operación, pero el bloque dispone de espacio.
*       - (1) la operación se ha realizado correctamente
*/

entero addRecord(fileRecord record, unifiedRecord structRecordFile);


/**
* Borra un registro del bloque
*
* @param idRecord: identificador del registro a borrar
*
* @pre: idRecord > 0
* @pos: si se encuentra el registro en el bloque, éste se borra de forma lógica
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha encontrado el registro
*       - (0) el registro ya está borrado
*       - (1) la operación se ha realizado correctamente
*/
entero deleteRecord(entero idRecord);

/**
* Borra un registro del bloque por clave
*
* @param clave: clave del registro a borrar
* @param structFileRecord: estructura de los registros del fichero
*
* @pre: clave.type==structFileRecord.tipoCampos[clave]
* @pos: si se encuentra el registro en el bloque, éste se borra de forma lógica
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha encontrado el registro
*       - (0) el registro ya está borrado
*       - (1) la operación se ha realizado correctamente
*/
entero deleteRecord(secuencia<bytes> clave, unifiedrecord structFileRecord);

/**
* Borra un registro del bloque que sea idéntico al pasado como argumento
*
```

```
* @param record: registro a borrar
* @param structFileRecord: estructura de los registros del fichero
*
* @pre: record.unifiedRecord==structFileRecord
* @pos: si se encuentra el registro en el bloque, éste se borra de forma lógica
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha encontrado el registro
*       - (0) el registro ya está borrado
*       - (1) la operación se ha realizado correctamente
*/
entero deleteRecord(fileRecord record, unifiedRecord structFileRecord);
```

Listing 10: Operaciones públicas de bloque

4.3 Operaciones privadas

Como operaciones privadas, he considerado realizar operaciones de busqueda en secuencia, es decir, sólo avanzando empezando siempre desde el principio del bloque. En definitiva, se usaría un iterador que comenzaría en el principio del bloque, lo recorrería entero y simplemente devolvería el registro coincidente

```
// operaciones privadas de bloque
private

/**
* Busca un registro por identificador de registro
*
* @param idRecord: identificador de registro
*
* @pre: idRecord<Bfr
* @pos: simplemente busca hacia delante, nunca retrocede.
* @ret: copia del registro encontrado, o en caso contrario, un registro vacío con identificador -1.
*/
fileRecord find(entero idRecord) const;

/**
* Busca un registro por clave de registro
*
* @param clave: clave del registro
*
* @pos: simplemente busca hacia delante, nunca retrocede.
* @ret: copia del registro encontrado, o en caso contrario, un registro vacío con identificador -1.
*/
fileRecord find(secuencia<bytes> clave) const;
```

Listing 11: Operaciones privadas de bloque

Para poder movernos por el bloque, debemos usar un elemento que nos provea de ese movimiento; usaremos un iterador que supondremos que, al crearlo siempre apunta al primer registro del bloque y que sólo permite avanzar hacia delante.

Por tanto, para poder ir hacia atrás, deberemos crear otro iterador. Puesto que no sé cómo provocar que, dentro de la clase *fileASF* no se puedan crear dos iteradores sobre el mismo bloque en la misma visita al mismo, dejaré recalcado que **no se debe crear dos iteradores sobre el mismo bloque puesto que en archivos secuenciales físicos sólo se realiza una pasada en cada ocasión**.

El iterador queda como sigue:

```
// Iterador de bloque
private:

class iterator{
    private:
    puntero p;

    public:
    /**
    * Crea un puntero apuntando al primer registro del bloque
    *
    * @pos: el registro al que apunta puede ser modificado
    */
    iterator();

    /**
    * Avanza un registro, es decir, avanza tanto como el tamaño del registro
    *
    * @ret: devuelve una copia del registro al que apuntaba
    *
    * @pos: en caso de avanzar después del último, el iterador toma el valor nullptr tras devolver el registro al que apuntaba.
    */
    fileRecord operator++();

    /**
    * devuelve una copia del registro al que apunta
    *
    * @ret: registro al que apunta
    */
    fileRecord operator*();

    /**
    * Destruye el iterador, liberando la memoria dinámica utilizada.
    */
    ~iterator();

    private:

    /**
    * Permite conocer si el iterador apunta al último registro; para ello, realiza cálculos con el identificador del registro al que apunta, el factor de bloqueo y el tamaño de registro.
    *
    * @ret: booleano que indica si apunta al último o no.
    */
```

```
    boolean isLastRecord() const;
}
```

Listing 12: Iterador de bloque

De nuevo, recalco que no es la mejor estructura posible, pero no he conseguido crear otra que la mejore. Aún así, su uso será el adecuado en la construcción del tipo de dato abstracto *fileASF*, pues eso sí lo permite.

5 TDA fileHeader

En principio, este tipo de dato no es necesario y, de hecho, sería desconocido para el usuario. El motivo de ello es el siguiente; cuando trabajamos con un fichero, la cabecera dispone de todos los datos del fichero inherentes al propio fichero y que son exclusivos de él. Por tanto, no tiene sentido crear un tipo de dato abstracto para la cabecera puesto que sus operaciones dependerán de las del fichero en sí.

Aún así, voy a crear una estructura de datos que me ayudará a trabajar con ella de forma más compacta en el modelaje del *TDA fileASF*; la llamaré *fileHeader* y su gestión se realizará en el tipo de dato que la contiene.

```
// Representación en memoria de una cabecera de fichero
estructura fileHeader {
    const entero tamHeader;
    entero numBlocks;
    entero numRecords;
    const unifiedRecord structRecord;
    const unifiedBlock structBlock;
}
```

Listing 13: Especificación de la estructura dela cabecera del fichero en memoria

Como podemos observar en el código superior, la estructura de los registros y de los bloques de memoria será constante, ya que trabajamos exclusivamente con bloques homogéneos cuyos registros disponen de una longitud fija.

Podemos llegar a preguntarnos el por qué de guardar *structRecord* y *structBlock* aquí, pero esto es totalmente necesario para poder gestionar los registros y boques una vez se encuentren fuera del fichero.

6 TDA fileASF

Estamos ya en disposición de modelar el tipo de dato abstracto *fileASF*, nuestro objetivo. Para ello, usaremos las estructuras trabajadas previamente.

6.1 Estructura en memoria

La estructura en memoria que se nos plantea en el guión es la siguiente:

```
// Representación en memoria de un fichero ASF
private:
```

```
estructura fileASF {
    secuencia<caracter> diskFilename;
    diskFile fileManager;
    fileHeader header;
    fileBlock chargedBlock;
    boolean changedHeader;
    boolean changedContent;
    entero numBlocksLastBlock;
}
```

Listing 14: Especificación de la estructura del fichero en memoria

Realmente, hay una pequeña modificación; esta es *numBlocksLastBlock* que representa el número de bloques que tiene el último bloque del fichero. Este atributo es una mejora, permite conocer en qué ocasión vamos a tener que añadir un bloque cuando añadamos o modifiquemos un registro.

El cálculo para esta gestión es secillo y simplemente depende del factor de bloqueo; eso sí, debemos gestionar otro atributo más. Aún así, creo que es una mejora considerable.

6.2 Operaciones privadas

A diferencia del tipo de dato abstracto *fileBlock*, considero que es mejor definir las operaciones privadas primero; ayudará a comprender la construcción que haremos de las operaciones públicas.

Resaltaré algunas de ellas:

- *fCloseAndOpen*: se encarga de cerrar y abrir el fichero para colocar el manejador de fichero en la posición inicial; es decir, presenta una utilidad bastante grande a la hora de buscar.

  Además, podríamos pensar que es *fClose* y a continuación *fOpen*; sin embargo, no es así, sino que no necesita cargar la cabecera en disco. Simplemente escribe el bloque en disco, cierra el fichero y lo vuelve a abrir sin tener que cargar la cabecera.

- *chargeNextblock*: es la encargada de cargar el siguiente bloque del fichero; para ello, debe escribir el bloque que hay cargado en disco para cargar el siguiente. Es una función útil que sirve para recorrer el fichero completo al reordenar.

- *copyOnFile*: tiene la obligación de añadir los registros que se pasan como argumento al fichero que se pasa como argumento; es una función exclusivamente creada para realizar la reordenación bloque a bloque.

- *goToLastBlock*: como dice su nombre, mueve el manejador de fichero a la primera posición del último bloque del fichero y lo carga en *chargedBlock*. Previamente, se ha escrito el bloque que había cargado en disco.

  En caso de identificarse que el bloque que hay cargado ya es el último, no se realiza la operación.

Si nos fijamos, en todas o casi todas las operaciones he considerado trabajar con un código de estado de la operación. La causa viene impuesta por la gestión de inconsistencias; considero que, si tratamos de realizar la gestión de la inconsistencia justo cuando ocurre, es más sencillo mantener el fichero de forma correcta.

```
// Operaciones privadas de fileASF
private:

/**
* Obtiene un manejador de fichero para el fichero pasado como argumento
*
* @param path: ruta del archivo que se quiere abrir.
*
* @pre: path debe existir en el sistema, en caso contrario da un error
* @pos: this.diskFilename = path
*        el archivo no es modificado aunque puede ser modificado con posterioridad.
*       no carga el primer bloque del archivo.
*       carga el contenido de la cabecera.
* @ret: devuelve un diskFile que apunta a la primera posición de memoria del primer bloque del fichero.
*/
diskFile fOpen(secuencia<caracter> path);

/**
* Cierra el fichero actual liberando la memoria dinámica usada.
*
* @pos: this.fileManager=nullptr
*
* ret: codigo de operación:
*       (-1) no se ha podido cerrar el fichero o no existe
*       (0) el bloque de memoria o la cabecera están modificados
*       (1) se ha cerrado correctamente
*/
entero fClose();

/**
* Cierra y abre el fichero actual para comenzar al principio.
*
* @pos: this.diskFilename = path
*        el archivo no es modificado aunque puede ser modificado con posterioridad.
*        fileManager se impone en la primera posición del primer bloque del fichero.
* ret: codigo de operación:
*       (-1) no se ha podido realizar la operación o hay inconsistencias.
*       (0) el bloque de memoria o la cabecera están modificados
*       (1) se ha cerrado y abierto correctamente, fileManager está en posición correcta.
*/
entero fCloseAndOpen();

/**
```

```
* Devuelve una copia del registro que tiene la clave pasada como argumento.
*
* @param clave: clave del registro que se busca
*
* @pre: clave tiene una estructura adecuada, siguiendo la que hay en header.structRecord.
* @pos: el registro podrá ser modificado.
*       Cierra y abre el fichero para comenzar a buscar desde el principio.
*
* ret: copia del registro que se ha pedido, en caso de no encontrarse se devuelve vacío.
*/
fileRecord findRecord(secuencia<bytes> clave);

/**
* Devuelve una copia del registro que es idéntico al pasado como argumento.
*
* @param record: registro que se busca.
*
* @pre: record tiene una estructura adecuada, siguiendo la que hay en header.structRecord.
* @pos: el registro podrá ser modificado.
*       Cierra y abre el fichero para comenzar a buscar desde el principio.
*
* ret: copia del registro que se ha pedido, en caso de no encontrarse se devuelve vacío.
*/
fileRecord findRecord(fileRecord record);

/**
* Escribe el contenido del bloque en memoria.
*
* @pre: changedContent==true;
* @pos: el bloque queda almacenado en memoria poniendo el flag changedContent a false.
*
* @ret: codigo de estado de la operación
*       (-1) no se ha podido escribir el bloque en memoria o hay alguna inconsistencia con la cabecera.
*       (0) el bloque no ha sido modificado.
*       (1) operación realizada correctamente.
*/
entero writeBlockOnDisk();

/**
* Escribe el contenido de la cabecera en memoria.
*
* @pre: changedHeader==true;
* @pos: la cabecera del fichero queda almacenada en memoria poniendo el flag changedHeader a false.
*
* @ret: codigo de estado de la operación
```

```
*        (-1) no se ha podido escribir la cabecera en memoria o hay alguna inconsistencia con el contenido.
*        (0) la cabecera no ha sido modificada.
*        (1) operación realizada correctamente.
*/
entero writeHeaderOnDisk();

/**
* Escribe todo el objeto en memoria.
*
* @pre: changedHeader==true and changedContent==true;
* @pos: fileheader y chargedBlock quedan almacenados en memoria con changedHeader=false=changedContent.
*
* @ret: codigo de estado de la operación
*        (-1) no se ha podido realizar la operación o hay alguna inconsistencia.
*        (0) ningún objeto ha sido modificado
*        (1) operación realizada correctamente.
*/
entero writeOnDisk();

/**
* Añade un registro a la cabecera
*
* @pos: changedHeader=true y header.numRecords+=1.
* @ret: codigo de estado de la operación
*        (-1) no se ha podido realizar la operación.
*        (1) operación realizada correctamente.
*/
entero addRecordOnHeader();

/**
* Sustrae un registro a la cabecera
*
* @pos: changedHeader=true y header.numRecords-=1.
* @ret: codigo de estado de la operación
*        (-1) no se ha podido realizar la operación.
*        (1) operación realizada correctamente.
*/
entero deleteRecordOnHeader();

/**
* Añade un bloque a la cabecera
*
* @pos: changedHeader=true
*        header.numBlocks+=1.
*
* @ret: codigo de estado de la operación
*        (-1) no se ha podido realizar la operación.
*        (1) operación realizada correctamente.
*/
entero addBlockOnHeader();

/**
* Obtiene una lista con una copia de todos los registros del bloque
```

```
    chargedBlock que no están modificados.
*
* @ret: secuencia de fileRecord con los registros no borrados de chargedBlock
*/
secuencia<fileRecord> getNonDeleted() const;

/**
* Copia la lista de registros pasada como argumento al final del fichero pasado como argumento.
*
* @param recordSequence: secuencia de registros a copiar
* @param newFile: fichero en el que copiarlos
*
* @pos: newFile cambia su contenido.
*/
void copyOnFile(secuencia<fileRecord> recordSequence, fileASF newFile);

/**
* Escribe el bloque chargedBlock en memoria si ha sido modificado y hay consistencia. Y carga el siguiente bloque (siguiente identificador) en chargedBlock. Si no hay ninguno cargado, carga el bloque en el que esté apuntando fileManager
*
* @pos: el contenido que había en chargedBlock se escribe en memoria.
*       chargedBlock tiene el contenido del siguiente bloque en memoria.
*       fileManager apunta al primer registro del bloque cargado.
*
* @ret: codigo de estado de la operación
*       (-1) no se ha podido realizar la operación.
*       (0) ya es el último bloque se ha creado un bloque nuevo y se ha cargado en chargedBlock.
*       (1) operación realizada correctamente.
*/
entero chargeNextBlock();

/**
* Avanza hasta el último bloque y lo carga en chargedBlock.
*
* @pos: se escribe el contenido de chargedBlock en disco antes de cargar el último bloque. Si está cargado el último bloque, no se escribe en disco.
*
* @ret: codigo de estado de la operación
*       (-1) no se ha podido realizar la operación.
*       (0) chargedBlock es el último bloque.
*       (1) operación realizada correctamente.
*/
entero goToLastBlock();
```

Listing 15: Operaciones privadas del fichero en memoria

6.3 Operaciones públicas

Como sabemos, debemos disponer de un constructor y de un destructor, al menos. En nuestro caso, disponemos de un constructor que trata de abrir el fichero cuya ruta es la pasada como argumento y, en caso de no encontrarlo, lo crea.

La creación del objeto simplemente incluye la inicialización de la cabecera, la colocación del descriptor de archivo en la primera posición de memoria del primer bloque del archivo y deja vacío el bloque a cargar.

Además, claramente disponemos de un destructor que liberará la memoria dínamica utilizada y escribirá en disco el contenido del objeto en caso de modificación.

```
// Constructor y destructor
public:

/**
* Crea un objeto abriendo el fichero pasado como argumento o creá
    ndolo
*
* @pre: path debe ser una ruta de un sistema de archivos.
* @pos: puede crear un fichero en el sistema de archivos,
    completamente vacío.
*       abre el fichero pasado como argumento.
*       fileManager apunta al primer registro del fichero.
*       chargedBlock está vacío hasta que se añada un registro.
*       se carga el contenido de la cabecera.
*
*/
fileASF(diskFilename path);

/**
* Destruye el objeto actual, escribiendo previamente en disco si
    hay modificaciones y son consistentes, en caso contrario, da un
    error.
*
* @pos: fileManager = nullptr
*       el objeto completo es destruido escribiendo en disco el
    contenido necesario.
*
*/
~fileASF();

}
```

Listing 16: Constructor y destructor del fichero

Como operaciones públicas tendremos las encargadas de leer un registro, reordenar el fichero, insertar un registro, borrar un registro y modificar un registro. Tratemos un poco cada una de ellas:

- *getRecord*: simplemente cierra y abre el fichero para buscar el registro de forma secuencial, leyendo el fichero completo en caso de no encontrarlo. En caso de encontrarlo, devuelve el registro en sí y, si no lo encuentra, devuelve un registro vacío.

- *reorder*: su función es obtener un nuevo fichero que albergue a todos los registros del fichero actual que no estén borrados; para ello, hará uso de las funciones privadas *getNonDeleted* y *chargeNextBlock*.

  Además, se realiza la gestión de la cabecera a medida que se va realizando la copia de los registros en el nuevo fichero, añadiendo las cantidades sobre registros y bloques que sean requeridas, mediante la función *insertRecord* aplicada al nuevo fichero ya creado.

- *insertRecord*: Se encarga de cerrar y abrir el archivo, modificando la cabecera para reflejar que se va a añadir el registro. A continuación, si no podemos insertar en el último bloque por cumplir el factor de bloqueo (usa *numRecordsLastBlock* para determinarlo), aumenta el número de bloques reservados, desplaza el organizador de fichero al último bloque para insertar un nuevo bloque y añadir el registro. En caso de que se pueda añadir al final, simplemente se desplazaría el organizador de fichero al final último bloque para añadir el registro, habiendo escrito previamente en la cabecera el aumento del número de registros. Esta operación dispone de una operación análoga para una secuencia de registros, el comportamiento es parecido, simplemente las operacione sque se pueden hacer de forma conjunta se hace. [4]

- *deleteRecord*: es la encargada de borrar un registro de forma lógica; se busca el registro en el fichero, con las operaciones privadas que eso conlleva; en caso de que se haya encontrado, se modifica el contenido de la cabecera del fichero adecuadamente (sin disminuir la cantidad de bloques reservados) y, después, se realiza el borrado lógico del registro. Al igual que ocurría con la operación anterior, disponemos de una operación para secuencias de registros; la lógica de actuación sería conocer primero cúantos de ellos s pueden borrar para modificar la cabecera y después ir borrándolos uno a uno.

- *modifyRecord*: se encarga de modificar el contenido de un registro; tal y como hemos tratado ya, no podemos modificar como tal el contenido, sino que debemos borrar el registro a modificar y añadir un nuevo registro.

  En un inicio, deberemos saber si el registro está dentro del fichero; para ello, lo buscaremos y, en caso de que sí exista realizaremos la modificación.

  Supuesto que el registro existe en el fichero, vamos a añadir un registro, entonces el factor de bloqueo nos dirá si debemos añadir un nuevo bloque, que en caso de ser necesario, se reflejará en la cabecera de forma adecuada. A continuación, realizamos el borrado lógico del registro y añadimos uno al final del archivo con el contenido modificado, sin modificar la clave del registro inicial.

  Cabe destacar que, en caso de apagón, si hemos ido modificando la cabecera en cada una de las operaciones que hemos realizado, podremos intuir o recuperar la consistencia de forma correcta.

```
// Operaciones públicas
```

[4] Es decir, como podemos calcular los bloques que nos harán falta y sabemos los registros que añadiremos podremos modificar una sola vez la cabecera.

```
public:

/**
* Devuelve una copia del registro con la clave pasada como argumento.
*
* @param clave : clave del registro a buscar
*
* @pre: la estructura de la clave debe ser la adecuada para el fichero
*
* @ret: copia del registro encontrado, en caso de no encontrarlo devolverá un registro vacío.
*/
fileRecord getRecord(secuencia<caracter> clave);

/**
* Devuelve un fichero completamente nuevo cuyo contenido es la reordenación de este.
*
* @ret: fichero reordenado, si todos están borrados, crea un nuevo fichero que dispone de un bloque de memoria reservado y completamente vacío.
*/
fileASF reorder();

/**
* Inserta un registro al final del documento, en caso de necesitarlo, añade otro bloque al fichero.
*
* @pre: record.structRecord==this.header.structRecord
* @pos: se carga el último bloque en chargedBlock, escribiendo el que había en memoria.
*       this.header.numRecords+=1
*       changedHeader=true
*       changedContent=true
*       numBlocksLastBlock modifica dependiendo de la casuística:
*           - Si Bfr>numBlocksLastBlock -> numBlocksLastBlock+=1
*           - Si Bfr<=numBlocksLastBlock -> numBlocksLastBlock=1 y this.header.numBlocks+=1
* @ret: código de operación:
*       (-1) no se ha podido realizar la operación o hay inconsistencias.
*       (1) se ha insertado correctamente.
*/
entero insertRecord(fileRecord record);

/**
* Inserta una secuencia de registros al final del documento, en caso de necesitarlo, añade los bloques necesarios al fichero.
*
* @pre: record[i].structRecord==this.header.structRecord
* @pos: se carga el último bloque en chargedBlock, escribiendo el que había en memoria de forma iterada.
*       this.header.numRecords es aumentado.
*       changedHeader=true
```

```
*        changedContent=true
*        numBlocksLastBlock y this.header.numBlocks se modifica siguiendo la lógica de la operación anterior.
* @ret: código de operación:
*        (-1) no se ha podido realizar la operación o hay inconsistencias.
*        (0) algún/unos registros no se han podido insertar.
*        (1) se han insertado todos los registros correctamente.
*/
entero insertRecordSequence(sequence<fileRecord> records);

/**
* Realiza un borrado lógico del registro con la clave pasada como argumento.
*
* @pre: la clave dispone del formato y tipo adecuado para el fichero.
* @pos: se carga el bloque del registro en chargedBlock, escribiendo el que había en memoria.
*        this.header.numRecords-=1
*        changedContent=true
*        changedHeader=true
*        numBlocksLastBlock-=1
* @ret: código de operación:
*        (-1) no se ha podido realizar la operación o hay inconsistencias.
*        (0) el registro ya se encuentra borrado o no se ha encontrado.
*        (1) se ha borrado correctamente.
*/
entero deleteRecord(secuencia<caracter> clave);

/**
* Realizar un borrado lógico de una secuencia de registros con las claves pasadas como argumento.
*
* @pre: la claves disponen del formato y tipo adecuado para el fichero.
*        claves.numElements>=this.header.numRecords
* @pos: se realizan las mismas modificaciones que en la operación anterior pero en escala.
* @ret: código de operación:
*        (-1) no se ha podido realizar la operación o hay inconsistencias.
*        (0) algún/unos de los registros ya se encuentran borrado o no se han encontrado.
*        (1) se han borrado todos correctamente.
*/
entero deleteRecordSequence(secuencia<secuencia<caracter>> claves);

/**
* Modifica el contenido de un registro sin cambiar su clave.
*
* @pre: record.structRecord==this.header.structRecord
* @pos: puede modificar la cabecera añadiendo (o no) un bloque (depende de la situación).
```

```
*        this.header.numRecords no se modifica.
*        la clave del registro no se permite su modificación.
* @ret: código de operación:
*        (-1) no se ha podido realizar la operación o hay inconsistencias.
*        (0) no se ha encontrado el registro o dispone del mismo contenido que el sugerido por la modificación.
*        (1) se ha modificado correctamente.
*/
entero modifyRecord(fileRecord record);

/**
* Modifica el contenido de una secuencia de registros sin cambiar sus claves.
*
* @pre: record[i].structRecord==this.header.structRecord
* @pos: puede modificar la cabecera añadiendo (o no) un(o varios) bloque/s (depende de la situación).
*        this.header.numRecords no se modifica.
*        las claves de los registros no son modificadas.
* @ret: código de operación:
*        (-1) no se ha podido realizar la operación o hay inconsistencias.
*        (0) no se ha encontrado algún/unos registro/s o dispone/n del mismo contenido que el sugerido por su modificación.
*        (1) se han modificado todos correctamente.
*/
entero modifyRecordSequence(secuencia<fileRecord> records);

}
```

Listing 17: Operaciones públicas del fichero

Es importante recalcar que, aunque no se han especificado completamente cómo son los desplazamientos necesarios de la estructura *fileManager*, si queremos movernos de registro en registro el *offset* será el tamaño del registro y, en caso de querer movernos por bloques, el *offset* será el tamaño de bloque.